

PyGA: A Python to FPGA Compiler Prototype

Yohann Uguen

Univ Lyon, INSA Lyon, Inria, CITI, ECR Lab.
France
yohann.uguen@insa-lyon.fr

Eric Petit

Intel Corporation, ECR Lab.
France
eric.petit@intel.com

Abstract

Field Programmable Gate Arrays, FPGAs, are a widely available configurable hardware design that is commonly used in many domain-specific applications. However, the complexity of its programming interface is currently restricting its usage to highly qualified programmers dedicated to FPGAs. In order to democratize FPGAs, many efforts are concentrating on High-Level Synthesis, HLS: the process of compiling a high-level language to hardware. In that context we propose PyGA, a proof of concept of a Python to FPGA compiler based on the Numba Just-In-Time (JIT) compiler for Python and the Intel FPGA SDK for OpenCL. It allows any Python user to use a FPGA card as an accelerator for Python seamlessly. As expected, early performance results are encouraging, but not competitive with compiled CPU version. The study shows that, to avoid overhead that cannot be compensated otherwise, tightly coupled accelerator design such as Intel Xeon+FPGA are necessary to address larger code base with finer grain kernel. It also shows that without FPGA-specific programming effort, HLS compilation and runtime efforts remain to be done to be competitive with modern multi-core CPUs.

CCS Concepts • **Hardware** → **Hardware accelerators; Reconfigurable logic applications;** • **Software and its engineering** → **Compilers;**

Keywords Hardware accelerators; High-level synthesis; Off-line Compiler

ACM Reference Format:

Yohann Uguen and Eric Petit. 2018. PyGA: A Python to FPGA Compiler Prototype. In *Proceedings of the 5th ACM SIGPLAN International Workshop on Artificial Intelligence and Empirical Methods for Software Engineering and Parallel Computing Systems (AI-SEPS '18)*, November 6, 2018, Boston, MA, USA. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3281070.3281072>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

AI-SEPS '18, November 6, 2018, Boston, MA, USA

© 2018 Association for Computing Machinery.

ACM ISBN 978-1-4503-6067-8/18/11...\$15.00

<https://doi.org/10.1145/3281070.3281072>

1 Introduction

Numerical computing and Artificial Intelligence (AI) based computations are more and more resource demanding. To handle such computations, efficient programming languages and frameworks are required. The current trend is to use Python based environments and libraries [1, 15, 20, 22] to develop and accelerate code. Such environments use compiled versions of the Python code that runs on the host CPU. These can be accelerated using standard high performance CPU libraries like Intel MKL and MKL-DNN. *Ad-hoc* libraries also allow the use of additional hardware like General Purpose Graphical Processors Units (GPGPUs) when beneficial.

Other options are available such as the use of Field Programmable Gate Arrays (FPGAs) which have been proven very efficient in terms of compute power [17] and energy consumption [25]. These are circuits with an undefined function at the time of manufacturing. They can be reconfigured through software as many times as necessary. The modularity of their architecture allows programmers to avoid usual CPUs and GPGPUs bottlenecks. FPGA programming models are based on very low-level and domain specific programming languages. It makes it difficult to democratize to a larger audience outside the traditional market for FPGAs and enthusiastic programmers. FPGA vendors also propose domain-specific library that can be use by any user, include from python codes, but it does not offer the customization capability that many programmers need. However FPGAs and tools to program them are becoming more accessible each day. For example, AWS offers FPGAs on demand using their Elastic Compute Cloud service.

This work is a proof of concept that Python code can be offloaded to a FPGA without any knowledge from the user about the FPGA technology. The objective is to provide Python developers an easy way to compile a bit-stream (the FPGA executable) that they can ship with their package. Thus High Performance Computing (HPC) or AI users could use a FPGA card as an accelerator seamlessly. We are focusing on scientific programming with Python, especially on Numpy [20] arrays manipulation in this first prototype. It also evaluates, on a simple benchmark, the impact of being tied to the PCIe interface to offload the computation, and illustrate state-of-art HLS capacities regarding kernels not specifically tuned for FPGAs.

The paper is organized as follows: Section 2 presents the state of the art of FPGA programming; Section 3 presents PyGA implementation and the tools it relies upon; Section 4

presents early results obtained to benchmark the approach and Section 5 concludes.

2 State of the Art

The usual way to program a FPGA is to write a circuit description using a Hardware Design Language (HDL). These are tedious to write and difficult to debug, making development costs orders of magnitude higher than classical programming. In an effort to make FPGAs programming easier, higher-level languages (such as C, OpenCL, Matlab, etc.) are now being considered as FPGA programming languages [19]. In opposition to HDL, these languages provide a behavioral description of a program. The process of compiling a program to hardware is called High-Level Synthesis (HLS).

The input source for HLS varies from one tool to another but most of them are C/C++. In a more accuracy-focused approach, FloPoCo [9] suggests to use mathematical specifications as inputs. Even though building a bitstream is easier by using HLS tools, it is still difficult to program a FPGA. Indeed, connecting an IP to a FPGA memory, pins, etc. requires strong knowledge about hardware programming. For example, in an HPC context, Intel is proposing the Intel FPGA SDK for OpenCL. These tools are fundamentally different as they do not require the user to integrate their IPs into HDL projects.

However, in the broader scientific community, a lot of applications are done using mathematical languages such as Matlab or Scilab, and Python with Scipy [15, 20]. Python also appears to be the winning language of the new AI computing wave [1, 22], often accelerated using JIT compilers such as Numba [18] or Cython [4] or google XLA. There is even large scale simulation based on Python arising like PyFR [28]. Using these Python-based languages, programmers mostly rely on compilers and runtimes to perform well. Such approaches are not suitable for a FPGA workflow as compilation times are usually very long (from hours to weeks). Therefore, using Python as a HLS input language would unleash the potential of FPGAs to HPC and AI programmers.

Listing 1. PyGA usage on a simple example

```
from Py2FPGA import PyGA
def __my_func(a, b):
    return a+b
res = PyGA(__my_func, [4, 5.3])
```

Some HLS tools already offers Python code as an input [6, 10] but only generates IPs that further needs to be integrated in a HDL project. It is also possible to transform Python functions to their C equivalent [4] and then use a C/C++ based HLS tool but this approach would suffer from the same integration problems as the latter. Another approach could have been to use OpenCL API for Python [16] combined with the Intel FPGA SDK for OpenCL but this would require users to write OpenCL code, the same way it would be done

with the C/C++ OpenCL interface. Therefore we present our tool, PyGA that runs Python functions on a FPGA without having to write any other code than Python.

3 PyGA: Implementation Details

The off-line compilation flow presented in this paper relies on two tools: Numba and the Intel FPGA SDK for OpenCL. These tools are shortly presented before detailing the concept of PyGA. As this is a proof of concept, PyGA implementation is very minimal.

Numba In order to accelerate Python code, one can use Numba [18] as a JIT compiler. Numba detects which functions to speedup by looking at users annotations. Numba translates a function code to its LLVM Internal Representation (IR); which can then be compiled to the host architecture, as illustrated by Figure 1 (left).

Intel FPGA SDK for OpenCL When programming a FPGA, one must: get an implementation of the design to run; interface it with the correct FPGA model pins; load and unload the memory of the FPGA using a Computer Aided Design (CAD) software. All these steps require strong knowledge about FPGAs and CAD tools. The Intel FPGA SDK for OpenCL [7] is a toolkit that hides these steps to the programmers.

As depicted by Figure 1 (right), the user needs to write an OpenCL kernel that implements the function to run on the FPGA, and a C++ host code that manages the FPGA. On one side, the kernel is compiled directly as a FPGA bit-stream, connecting to the correct pins. On the other side, a set of specific C++ functions allow the user to configure the FPGA, load its memory, launch the kernel and retrieve the result.

PyGA When using PyGA (illustrated by Figure 1) the user calls his function with the proper arguments without annotations such as in the toy example of Listing 1. The tool then uses Numba's type inference to get the function's signature and retrieves the Numba generated LLVM IR (Listing 2).

The Numba generated LLVM IR does not embed any parallelism nor specific FPGA optimization. Indeed, in a CPU context, this IR is supposed to run through optimizing compilation passes such as *llvm-opt*. These optimization passes exist in a FPGA context but have to be driven with goals such as: low latency; high frequency; low resource usage.

An OpenCL wrapper is then generated with the correct arguments and types using the function's signature (Listing 3). From the wrapper, we retrieve the associated LLVM IR. This way, the Intel FPGA SDK for OpenCL generates the correct environment metadata. We then integrate the core of the Numba LLVM IR function. The two tools does not share the same LLVM IR version making us translate Numba's IR to Intel's one. Plus, Intel's IR uses a custom version of LLVM requiring a bit more translation. The modified wrapper's LLVM IR can then be compiled to a FPGA bit-stream and programmed to the FPGA.

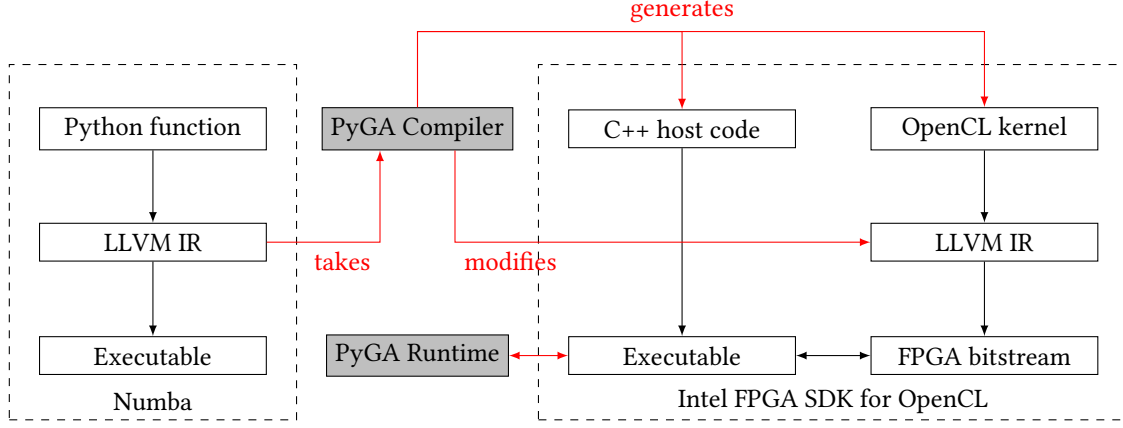


Figure 1. PyGA compilation flow

Listing 2. Numba generated LLVM IR from Listing 1

```

...
define i32 @"_ZN8__main__13__my_func$241Edd"
(double* noalias nocapture %retptr, ...,
double %arg.a, double %arg.b) ... {
entry:
    %.15 = fadd double %arg.a, %arg.b
    store double %.15, double* %retptr, align 8
    ret i32 0
}
...

```

Meanwhile, the host code is generated using the function’s signature. This consists of: FPGA initialization functions, memory allocations on the FPGA for the Python data; events management for memory transfers and kernel launch; results retrieval. This is all done using the Intel FPGA SDK for OpenCL API and usual OpenCL functions. The kernel is configured with a single work-group and work-item kernel. This code is compiled to a shared object in order to allow Python and the host code to share the same address space. Finally, Python is calling the host code to run the program.

The current implementation allows us to off-load simple Python functions to a FPGA. However this early prototype currently presents several limitations. During the kernel execution, there cannot be interactions with the Python (e.g exceptions). One can only use scalar values or Numpy arrays. Plus, the Numpy arrays must be manipulated element by element. These last are strong limitations due to the current state of the implementation that could be lifted in the future without strong technical limitation.

Listing 3. OpenCL wrapper for example from Listing 1

```

__kernel void pyfunc(
__global const double * restrict a,
__global const double * restrict b,
__global double * restrict retptr){}

```

This prototype has been functionally tested with several simple examples. The results were validated against the Python interpreter results.

4 Early Performance Evaluation

For this first prototype, we had quite low expectation on performance since we did not modify our benchmarks to specifically match the FPGA requirements. Without careful programming, current HLS compilers are generally struggling to make FPGAs compete with CPUs and GPUs. As we rely on such tools, they are on average providing us the upper bound of the performance we can reach (the same can be said to automatically addressed GPGPU accelerators from C or Fortran code).

4.1 Deriche Edge Detector

The Deriche edge detector [12] is a multi-step algorithm that detects contours on a given image. The input image is scanned in both horizontal directions to compute two new images where each point results from a linear combination of its neighbors. These two images are summed to create a third one. Another similar pass is done following the vertical axis. Each pixel of the resulting image then goes through an activator function to determines if the final pixel color.

We implemented this algorithm in Python and compared the execution time of: the Python interpreter; the Numba JIT on CPU; and PyGA on FPGA. The experiments were run on a dual sandy bridge Xeon 8C/16T @2.60GHz with an Intel Programmable Acceleration Card Arria 10 (PAC A10) FPGA on a 12000*6000 pixels image. The results we obtained are given in Table 1. As expected the bit-stream compilation is very long, motivating this work as an off-line compiler.

The Pyga results are decomposed in different time categories. *Platform initialization* must be done once by the Python interpreter and therefore is not significant for larger applications. *Memory allocation* and *memory transfers* are

Table 1. Execution time (in seconds) of the Deriche edge detector implemented in Python.

		Deriche
Python Interpreter		~3460
Numba JIT	Compilation	~4
	Execution	~3
PyGA compiler	Bitstream compilation	~18336
	Host code compilation	<1
	Program FPGA	~3
PyGA runtime	Platform initialization	~2
	Allocate FPGA mem	<1
	Mem tranfert to FPGA	<1
	Kernel only	~24
	Mem transfert to host	<1

all together taking almost 3 seconds. If we cannot avoid such overheads, the transfer time already matches the kernel execution time on CPU despite the very reasonable kernel granularity. This is calling for a better integration of the FPGA into the memory system of the CPU.

Finally the bit-stream generated by the HLS tool from our input is not competitive with CPU. The PyGA kernel is nearly 8x (24 to 3s) slower than the Numba JIT version on CPU but still 125x (3460 to 24s) faster than the Python interpreter. These results are mainly explained by the fact that the generated hardware is a single task running through the image. It would largely benefit from hardware parallelization and compiler hints regarding loops performance which are not implemented yet. It would also be interesting to compare this result with an handcrafted VHDL version of the kernel.

All in all this is encouraging as the produced code is not optimized at all. For this work to be relevant in a real world context, some improvements must be made. These fall in two categories developed in the following future work Section; compile-time optimizations and run-time optimizations.

4.2 Future Work

4.2.1 PyGA Compiler Optimizations

Adaptation of Numba: The LLVM IR generated by Numba is targeting a CPU execution with a unified memory space. Therefore, a mapping of pointer variable and address space has to be computed. Numba might also generate overly complicated code, which is problematic when compiling for a FPGA as all the logic will be synthesized and therefore use resources and potentially slow down the execution.

FPGA Aware Compiler Optimization: Current HLS tools relies heavily on FPGA friendly implementation to be efficient. Most of the time this is based on user compiler directives. However for general purpose kernel acceleration this process can be largely automated. Compiler optimization such as the Polyhedral model for loop transformation is a very interesting option to consider [3].

Custom Length Data-Types: One of FPGAs best advantages is that they can (should!) use custom length data-types.

This allows for resource saving, possibly reaching higher frequencies, and less energy consumption. Restricting users to CPU friendly data-types of 16, 32 or 64 bits does not make sense in a FPGA context. It exists various ways to assess the precision of a computation that can be tuned to provide hints of proof to specialize a kernel with a custom type [9, 11, 26, 27].

Code Specialization: Once a kernel is generated for the FPGA, we cannot easily reinterpret or recompile it for different input sizes. We then have to capture the best fitting input size(s). Alternatively we could decompose the input data in the most suitable block size to specialize data accesses and loop bounds. Depending on the available FPGA logic and the best code version for the actual input data, we must generate a guard and decision tree to select the best specialization [8, 13, 23].

4.2.2 PyGA Run-Time Optimizations

The FPGA must be programmed once the correct set of bit-streams are compiled with PyGA. However, beyond trivial cases, it may not be possible to put all the kernels and their potential specialized version at once on the FPGA logic. However, FPGA partial reconfiguration [21] will allow us to reconfigure it on the fly at a reasonable cost. Building the set of kernels to load is an interesting scheduling problem which need to be integrated into the PyGA runtime. Similar work can be found for JIT in constrained environments [5, 13], Systems-on-Chip [24] or GPGPUs [2, 14]. Furthermore, the usage of preexisting math library for FPGAs or finely tuned FPGA implementations provided by the user must be integrated into the scheduler as potential candidate for mapping on FPGA during partial reconfiguration. Finally, because we are using Numba, we can trivially integrate the CPU-compiled version of the kernel as a fall-back version.

5 Conclusion

In this paper we explored the opportunity to accelerate compute intensive kernels in interpreted languages through HLS tools. Our prototype, targeting Python and more specifically Numpy arrays in Scipy, allowed us to expose the concept and prove its feasibility. The proposed idea is not to JIT-compile the kernels, but to generate offline FPGA bit-streams for the application kernel and to call them at runtime during the Python application execution.

Early performance results emphasis a critical aspect of using FPGAs to accelerate kernels in general purpose computations. Current HLS compilers are not designed to implement the required heuristics and optimizations to transform general kernels for FPGAs, even fully defined structured nested loops. Therefore we also provide in our future-work section the list of required development and essential optimization to work on to make this project a competitive solution.

Acknowledgments

This project has been funded through the ECR Lab. by CEA-DAM, Intel, and UVSQ. Special thanks to John Freeman and Tommy Hoffner, Othman Bouizi, and Marc Gaucheron from Intel all over the world who gave us help, support and useful feedback anytime we needed it.

References

- [1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. 2016. TensorFlow: A System for Large-scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI'16)*. USENIX Association, Berkeley, CA, USA, 265–283.
- [2] C. Augonnet, S. Thibault, R. Namyst, and P.-A. Wacrenier. 2011. StarPU: a unified platform for task scheduling on heterogeneous multicore architectures. *Concurrency and Computation: Practice and Experience* 23, 2 (2011), 187–198.
- [3] M. M. Baskaran, J. Ramanujam, and P. Sadayappan. 2010. Automatic C-to-CUDA Code Generation for Affine Programs. In *Compiler Construction*, Rajiv Gupta (Ed.). Springer Berlin Heidelberg.
- [4] S. Behnel, R. Bradshaw, C. Citro, L. Dalcin, D. S. Seljebotn, and K. Smith. 2011. Cython: The Best of Both Worlds. *Computing in Science Engineering* (2011).
- [5] H.-P. Charles, D. Couroussé, V. Lomüller, F. A. Endo, and R. Gauguey. 2014. deGoal a tool to embed dynamic code generators into applications. In *International Conference on Compiler Construction*. Springer, 107–112.
- [6] R. Cieszewski, K. Pozniak, and R. Romaniuk. 2014. Python Based High-Level Synthesis Compiler. In *Proceedings of SPIE - The International Society for Optical Engineering*, Vol. 9290.
- [7] T. S. Czajkowski, U. Aydonat, D. Denisenko, J. Freeman, M. Kinsner, D. Neto, J. Wong, P. Yiannacouras, and D. P. Singh. 2012. From OpenCL to high-performance hardware on FPGAs. In *22nd International Conference on Field Programmable Logic and Applications (FPL)*.
- [8] F. Dang, H. Yu, and L. Rauchwerger. 2001. The R-LRPD test: Speculative parallelization of partially parallel loops. In *International Parallel and Distributed Processing Symposium*. IEEE.
- [9] F. de Dinechin and B. Pasca. 2011. Designing Custom Arithmetic Data Paths with FloPoCo. *IEEE Design & Test of Computers* (2011), 18–27.
- [10] J. Decaluwe. 2004. MyHDL: A Python-based Hardware Description Language. *Linux J.* 2004, 127 (2004).
- [11] Christophe Denis, Pablo de Oliveira Castro, and Eric Petit. 2016. Verificarlo: checking floating point accuracy through Monte Carlo Arithmetic. In *Computer Arithmetic (ARITH), 23rd Symposium on*. IEEE, 55–62.
- [12] R. Deriche. 1987. *Using Canny's criteria to derive a recursively implemented optimal edge detector*. Vol. 1. 167–187 pages.
- [13] A. Gal, B. Eich, M. Shaver, D. Anderson, D. Mandelin, M. R. Haghighat, B. Kaplan, G. Hoare, B. Zbarsky, J. Orendorff, et al. 2009. Trace-based just-in-time type specialization for dynamic languages. *Sigplan Notices* 44, 6 (2009), 465–478.
- [14] V. J. Jiménez, L. Vilanova, I. Gelado, M. Gil, G. Fursin, and N. Navarro. 2009. Predictive runtime code scheduling for heterogeneous architectures. In *International Conference on High-Performance Embedded Architectures and Compilers*. Springer, 19–33.
- [15] E. Jones, T. Oliphant, P. Peterson, et al. 2001. SciPy: Open source scientific tools for Python. <http://www.scipy.org/>
- [16] A. Klöckner, N. Pinto, Y. Lee, B. Catanzaro, P. Ivanov, and A. Fasih. 2012. PyCUDA and PyOpenCL: A Scripting-Based Approach to GPU Run-Time Code Generation. *Parallel Comput.* 38, 3 (2012), 157–174.
- [17] I. Kuon, R. Tessier, and J. Rose. 2007. FPGA Architecture: Survey and Challenges. (2007), 135–253.
- [18] S. K. Lam, A. Pitrou, and S. Seibert. 2015. Numba: A LLVM-based Python JIT Compiler. In *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC (LLVM '15)*. ACM, New York, NY, USA, 7:1–7:6.
- [19] R. Nane, V. M. Sima, C. Pilato, J. Choi, B. Fort, A. Canis, Y. T. Chen, H. Hsiao, S. Brown, F. Ferrandi, J. Anderson, and K. Bertels. 2016. A Survey and Evaluation of FPGA High-Level Synthesis Tools. *Computer-Aided Design of Integrated Circuits and Systems* (2016).
- [20] T. E. Oliphant. 2015. *Guide to NumPy* (2nd ed.). CreateSpace Independent Publishing Platform, USA.
- [21] K. Papadimitriou, A. Dollas, and S. Hauck. 2011. Performance of Partial Reconfiguration in FPGA Systems: A Survey and a Cost Model. *ACM Transactions on Reconfigurable Technology and Systems (TRETS)* 4, 4 (2011).
- [22] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. 2011. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12 (2011), 2825–2830.
- [23] E. Petit and F. Bodin. 2010. Code-Partitioning for a Concise Characterization of Programs for Decoupled Code Tuning. (2010). working paper or preprint.
- [24] E. Petit, G. Papaure, F. Dru, and F. Bodin. 2006. Astex: a hot path based thread extractor for distributed memory system on a chip. In *1st HiPEAC Industrial Workshop*. isbn–54.
- [25] J. Shalf, S. Dosanjh, and J. Morrison. 2011. Exascale Computing Technology Challenges. In *Proceedings of the 9th International Conference on High Performance Computing for Computational Science (VECPAR'10)*.
- [26] Devan Sohler, Pablo De Oliveira Castro, François Févotte, Bruno Lathuilière, Eric Petit, and Olivier Jamond. 2018. Confidence Intervals for Stochastic Arithmetic. (July 2018). <https://hal.archives-ouvertes.fr/hal-01827319> working paper or preprint.
- [27] Y. Uguen, F. de Dinechin, and S. Derrien. 2017. Bridging high-level synthesis and application-specific arithmetic: The case study of floating-point summations. In *2017 27th International Conference on Field Programmable Logic and Applications (FPL)*.
- [28] F.D. Witherden, A.M. Farrington, and P.E. Vincent. 2014. PyFR: An open source framework for solving advection–diffusion type problems on streaming architectures using the flux reconstruction approach. *Computer Physics Communications* 185, 11 (2014).